ABSTRACT

The Smart Plant Watering System Using Sensor Technology is an embedded and IoT-enabled irrigation prototype developed to automate plant watering based on real-time soil and environmental conditions. The system uses an ESP32 microcontroller as the central processing and communication unit. A soil-moisture sensor measures the moisture condition of the soil, a DHT11 sensor monitors ambient temperature and humidity, and a water-level sensor checks the availability of water in the storage tank. A relay-driven DC water pump supplies water to the plant whenever the soil moisture falls below the configured threshold. To prevent frequent switching, hysteresis control is used: the pump is switched ON below 35% soil moisture and switched OFF once the moisture reaches 55% or above. A dry-run protection mechanism disables the pump whenever the water tank is empty, while an additional 30-second maximum continuous runtime acts as a safety backup.

The prototype also includes a 16×2 I²C LCD for local monitoring and an ESP32-hosted local web dashboard for wireless monitoring without requiring internet access. The ESP32 creates an access point named SmartWatering and serves the dashboard at its local IP address. During startup, the LCD displays the Wi-Fi details and dashboard IP until a user opens the dashboard, after which the display cycles through soil moisture, estimated pH, temperature, humidity, tank status, pump state, and soil condition. Due to prototype cost constraints, a dedicated soil-pH probe is not used; instead, the displayed pH is clearly treated as a demonstration estimate derived from predefined moisture ranges and is not presented as a measured pH value. Experimental testing confirmed reliable sensing, automatic pump operation, tank-empty protection, LCD feedback, and responsive local web monitoring. The project provides a low-cost and practical platform for home gardens, nurseries, educational demonstrations, and small-scale smart irrigation applications.

CONTENTS

CHAPTER 1 INTRODUCTION

CHAPTER 2 SYSTEM DESIGN AND OPERATION

CHAPTER 3 HARDWARE DESIGN AND DESCRIPTION

CHAPTER 4 HARDWARE IMPLEMENTATION

CHAPTER 5 SOFTWARE DESCRIPTION

CHAPTER 6 SOFTWARE IMPLEMENTATION

CHAPTER 7 EXPERIMENTAL RESULTS

CHAPTER 8 APPLICATIONS, ADVANTAGES AND DISADVANTAGES

CHAPTER 9 CONCLUSION AND FUTURE SCOPE

REFERENCES

CHAPTER 1

INTRODUCTION

1.1 Overview

Efficient irrigation is essential for healthy plant growth, but conventional manual watering depends heavily on human attention and judgement. Plants may receive too much water, too little water, or water at the wrong time. These problems become more noticeable when the user is busy, away from home, or managing several plants. Excess watering wastes water and can damage roots, while insufficient watering causes wilting and poor growth. A smart plant watering system addresses these issues by combining sensors, embedded control, and automatic actuation so that watering decisions are based on actual soil and tank conditions rather than a fixed schedule.

The implemented system uses an ESP32 as the main controller. It reads a soil-moisture sensor, DHT11 temperature and humidity sensor, and an analog water-level sensor. The controller converts raw sensor readings into meaningful status values and automatically drives a DC water pump through a relay module. The pump is activated only when the soil is sufficiently dry and water is available in the tank. A 16×2 I²C LCD gives local feedback, while the built-in Wi-Fi capability of the ESP32 is used to create a local dashboard that can be opened from a phone or laptop through the displayed IP address.

The project is designed as a low-cost educational prototype. The final implementation includes soil-moisture monitoring, automatic irrigation, water-tank monitoring, temperature and humidity measurement, LCD monitoring, local Wi-Fi monitoring, pump dry-run protection, and a safety timeout. An estimated pH indicator is displayed only as a prototype demonstration parameter. It is derived from moisture ranges and is explicitly identified as estimated because no dedicated pH sensor is connected.

1.2 Problem Statement

Manual plant watering is inconsistent because it relies on the user to remember watering schedules and to estimate soil condition by observation. This can result in water wastage, overwatering, underwatering, and unnecessary human effort. In addition, a pump-based irrigation system can be damaged if it runs when the tank is empty. A low-cost system is therefore required that can continuously sense the soil condition, automatically provide water only when needed, stop watering when adequate moisture is reached, and prevent pump operation when the water source is unavailable. The system should also provide easily understandable local and wireless status information.

1.3 Objectives

- To design and implement a low-cost automatic plant watering system using an ESP32 microcontroller.
- To measure soil moisture continuously and convert the raw ADC value into an understandable percentage.
- To measure ambient temperature and humidity using the DHT11 sensor.

- To monitor the water available in the storage tank using an analog water-level sensor.
- To switch the irrigation pump automatically using a relay based on soil moisture and tank availability.
- To implement hysteresis control so that the pump turns ON below 35% moisture and turns OFF at 55% or above.
- To provide dry-run protection by keeping the pump OFF when the tank is empty.
- To implement a maximum continuous pump runtime of 30 seconds as an additional safety feature.
- To display live information on a 16×2 I²C LCD.
- To provide a local ESP32 web dashboard that can be opened through an IP address without internet access.
- To demonstrate an estimated pH indicator while clearly distinguishing it from real pH measurement.

1.4 Literature Survey

Earlier smart-irrigation studies have shown that sensor-based irrigation can reduce unnecessary watering by using soil and environmental information as the basis for control decisions. Automated irrigation systems commonly combine a microcontroller, moisture sensing, a water-control actuator, and a communication interface. Wireless sensor-network approaches extend this concept by allowing remote observation of field conditions and irrigation status.

Research on irrigation scheduling also highlights the importance of selecting appropriate sensing parameters and thresholds. Soil and plant conditions vary with crop type, soil texture, climate, and sensor placement. Therefore, calibration is essential for any practical prototype. In the present project, the soil-moisture sensor was experimentally calibrated using a dry reading of approximately 4095 and a wet reference of approximately 1500. The resulting percentage is used only as a prototype control scale, but it allows repeatable switching behaviour during demonstration.

Recent smart-farming systems frequently use Wi-Fi-capable microcontrollers such as ESP32 because they integrate analog and digital interfaces with wireless communication in one low-cost board. This makes it possible to combine real-time control with web-based monitoring. The present design follows this approach by hosting a dashboard directly on the ESP32, avoiding a dependency on cloud services or internet connectivity during project demonstration.

1.5 Existing and Proposed System Summary

In an existing manual watering arrangement, the user checks the plant and supplies water by hand. There is no continuous measurement of soil moisture, no protection against an empty water tank, and no automatic feedback. Timer-based watering improves automation but still applies water according to time rather than actual soil condition. Such systems may therefore irrigate unnecessarily after rainfall or fail to provide enough water under unusually dry conditions.

The proposed system uses actual sensor feedback. The ESP32 reads soil moisture, temperature, humidity, and water level, then makes an automatic watering decision. A relay isolates the controller from the pump switching path. The LCD presents local information, and the ESP32 Wi-Fi access point provides a live

monitoring dashboard. The use of separate ON and OFF thresholds prevents rapid relay switching near a single threshold, making the prototype more stable.

1.6 Hardware and Software Components Used

HARDWARE COMPONENTS

- ESP32 DevKit V1
- Capacitive/analog soil-moisture sensor
- DHT11 temperature and humidity sensor
- Analog water-level sensor
- 16×2 LCD with I²C interface
- Single-channel relay module
- DC water pump
- Water tank/reservoir
- Flexible water tube
- Plant pot and soil
- Suitable regulated power supplies
- Jumper wires, connectors and mounting base

SOFTWARE COMPONENTS

- Arduino IDE
- C/C++ embedded firmware
- Wire.h
- LiquidCrystal_I2C.h
- DHT.h
- WiFi.h
- WebServer.h
- HTML, CSS and JavaScript embedded in the ESP32 firmware

1.7 Organization of Report

Chapter 2: System Design and Operation describes the overall architecture, block diagram and working mechanism.

Chapter 3: Hardware Design and Description explains the ESP32, sensors, LCD, relay, pump and associated hardware.

Chapter 4: Hardware Implementation presents the circuit connections, pin mapping, mounting and integration details.

Chapter 5: Software Description explains Arduino IDE, firmware libraries, Wi-Fi access point and web dashboard technologies.

Chapter 6: Software Implementation presents the flowchart, control algorithm, thresholds and safety logic.

Chapter 7: Experimental Results presents the measured sensor values and successful operating test cases.

Chapter 8: Applications, Advantages and Disadvantages discusses practical uses, benefits and limitations.

Chapter 9: Conclusion and Future Scope summarizes the completed prototype and possible enhancements.

1.8 Summary

This chapter introduced the motivation for automatic watering, defined the problem addressed by the project, listed the objectives, summarized related smart-irrigation concepts, compared manual and sensor-based approaches, and identified the hardware and software used in the implemented prototype. The following chapter presents the detailed system architecture and operation.

CHAPTER 2

SYSTEM DESIGN AND OPERATION

This chapter describes the overall architecture and working mechanism of the Smart Plant Watering System. The design follows a sensing-processing-actuation-monitoring structure. Sensor inputs are acquired by the ESP32, converted into meaningful values, evaluated against control thresholds, and used to operate the pump through a relay. Information is displayed locally on the LCD and wirelessly through the ESP32 web dashboard.

2.1 Block Diagram

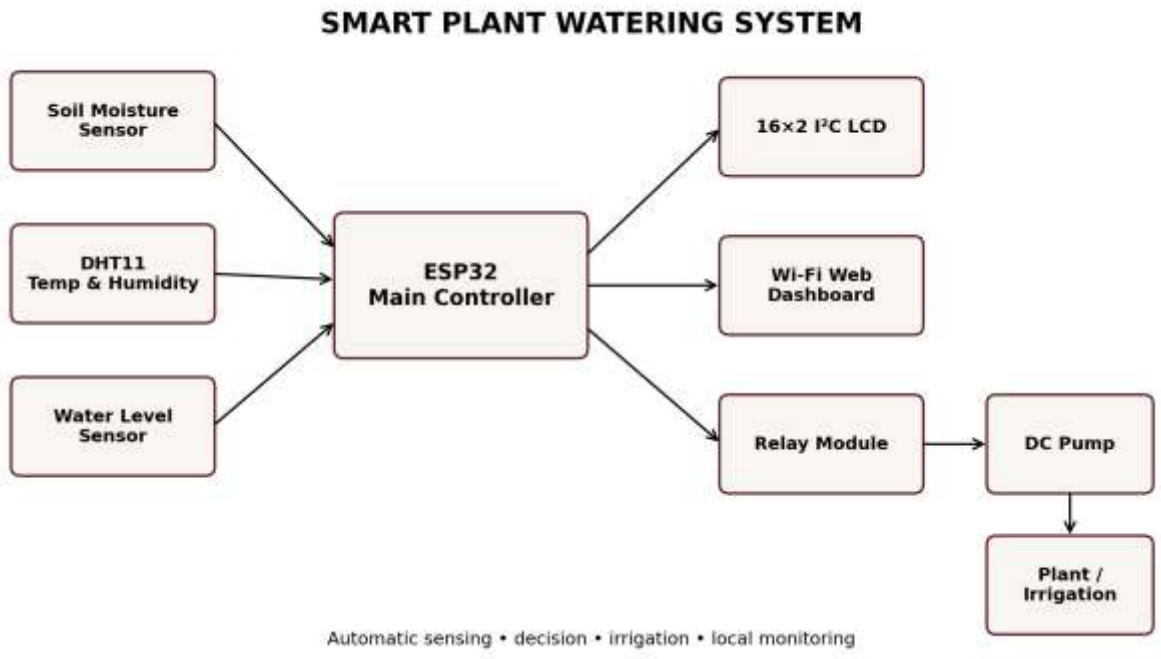


Figure 2.1 Block diagram of Smart Plant Watering System

2.2 Working Mechanism

The soil-moisture sensor is connected to GPIO 34 of the ESP32 and provides an analog value. During testing, the dry condition produced a value of 4095 while wet soil produced values in the approximate range of 1300 to 1900. For stable prototype operation, the firmware uses 4095 as the dry calibration point and 1500 as the wet calibration point, then maps this range to 0–100% moisture.

The DHT11 sensor is connected to GPIO 4 and supplies ambient temperature and relative humidity. The water-level sensor is connected to GPIO 35 and produces a low reading when dry and a higher value when immersed. Experimentally, a dry sensor produced 0, residual water droplets produced readings around 1040–

1175, and deeper immersion produced approximately 2200–2665. These readings were used to define the tank states EMPTY, LOW, MEDIUM and HIGH.

Automatic irrigation is based on hysteresis. If the tank is not empty and soil moisture falls below 35%, the ESP32 activates the relay and switches the pump ON. The pump remains active while the soil is below the OFF threshold. When moisture reaches 55% or more, the pump is switched OFF. If the tank becomes empty at any time, the pump is immediately disabled. A maximum continuous pump runtime of 30 seconds provides a second level of protection if the moisture reading fails to rise as expected.

The 16×2 I²C LCD uses GPIO 21 for SDA and GPIO 22 for SCL. During startup, the LCD shows the Wi-Fi network name and dashboard IP address. The ESP32 creates a local access point named SmartWatering. A phone or laptop can connect directly to this network and open the dashboard using the IP address, normally 192.168.4.1 in ESP32 access-point mode. Until the dashboard is opened, the IP remains visible on the LCD. After the first browser connection, the LCD begins rotating through live sensor and system-status pages.

2.3 Functional Requirements

- Continuous soil-moisture acquisition and percentage conversion.
- Temperature and humidity monitoring.
- Water-tank status classification.
- Automatic pump operation with separate ON and OFF thresholds.
- Pump inhibition when the water tank is empty.
- Maximum runtime safety shutdown.
- Live local LCD feedback.
- Live local web dashboard without internet dependency.
- Clear indication that the pH value is estimated and not sensor-measured.

CHAPTER 3

HARDWARE DESIGN AND DESCRIPTION

This chapter describes the major hardware modules used in the implemented prototype. The components were selected based on cost, availability, ease of interfacing with the ESP32, and suitability for a college demonstration model.

3.1 ESP32 DevKit V1

The ESP32 DevKit V1 is the central controller of the system. It provides a 32-bit dual-core microcontroller, multiple ADC-capable GPIO pins, I²C communication, and integrated 2.4 GHz Wi-Fi. In this project, the ESP32 reads all sensors, calculates the moisture percentage and estimated pH, executes the automatic watering logic, controls the relay, updates the LCD, creates the local Wi-Fi network, serves the web dashboard, and provides serial debugging output. GPIO 34 and 35 are used for analog sensing, GPIO 4 for DHT11 data, GPIO 26 for relay control, and GPIO 21/22 for the I²C LCD.

3.2 Soil-Moisture Sensor

The soil-moisture sensor is used to estimate the amount of moisture present around the plant roots. Its analog output changes with the electrical properties of the surrounding medium. The sensor is powered from 3.3 V and its analog output is connected to GPIO 34. The project calibration uses a dry reference of 4095 and a wet reference of 1500. These endpoints are mapped to 0–100% moisture. The sensor should be inserted only to the intended sensing depth and should be fixed firmly so that movement does not cause abrupt changes during demonstration.

3.3 DHT11 Temperature and Humidity Sensor

The DHT11 is a low-cost digital sensor that provides ambient temperature and relative humidity. It contains an internal sensing element and digital interface and requires only one data pin in addition to power and ground. In this prototype, DATA is connected to GPIO 4. Typical test readings during integration were approximately 28.9–29.3 °C and 78–80% relative humidity. The firmware also checks for invalid DHT readings so that a sensor failure can be identified.

3.4 Water-Level Sensor

The analog water-level sensor is installed in the tank to determine whether sufficient water is available for irrigation. Its signal is connected to GPIO 35. The project classifies the raw value as EMPTY below 200, LOW below 1200, MEDIUM below 2000, and HIGH at or above 2000. The sensor is used primarily as a discrete tank-status detector rather than as a precision volume meter. Keeping the pump OFF when the value indicates EMPTY prevents dry running.

3.5 16×2 I²C LCD Display

The 16×2 LCD provides local status information. An I²C backpack reduces the interface to four connections: VCC, GND, SDA and SCL. In the implemented system, the address is 0x27, SDA is connected to GPIO 21 and SCL to GPIO 22. The display shows startup status, Wi-Fi network details, the dashboard IP address, moisture, estimated pH, temperature, humidity, tank state, pump state and soil condition.

3.6 Relay Module

The relay module provides electrical switching between the low-power control side and the pump power circuit. The relay input is driven by GPIO 26. The implemented module operates with active-LOW logic: GPIO LOW activates the relay and GPIO HIGH deactivates it. The pump supply positive line is routed through the relay COM and NO contacts so that the pump is normally disconnected when the relay is inactive.

3.7 DC Water Pump and Tubing

The DC water pump moves water from the storage tank to the plant through flexible tubing. The pump is powered from a suitable external supply and is not powered directly from an ESP32 GPIO. The relay switches the pump supply. During demonstration, the tubing is positioned so that water reaches the soil while avoiding the electronics mounted on the baseboard.

3.8 Water Tank and Plant Pot

The water tank stores the irrigation water and supports the water-level sensor and pump. The plant pot contains the soil, moisture sensor and irrigation outlet. For safe mounting, the tank and plant are positioned away from the controller and relay wiring so that accidental water spills are less likely to reach the electronics.

3.9 Power Supply and Interconnections

The ESP32 and sensors require regulated low-voltage DC power. The pump and relay must be supplied according to their ratings. All logic-side modules that communicate directly with the ESP32 share a common reference ground where required. Jumper wires, terminal connections and cable ties are used to keep the prototype organized and to reduce accidental disconnection during demonstration.

CHAPTER 4

HARDWARE IMPLEMENTATION

The hardware implementation integrates all sensing, control, display, communication and pump-driving components into one working prototype. The final arrangement is mounted on a plywood base so that the plant, tank, LCD and electronic modules are visible and accessible during demonstration.

4.1 Circuit / Wiring Diagram

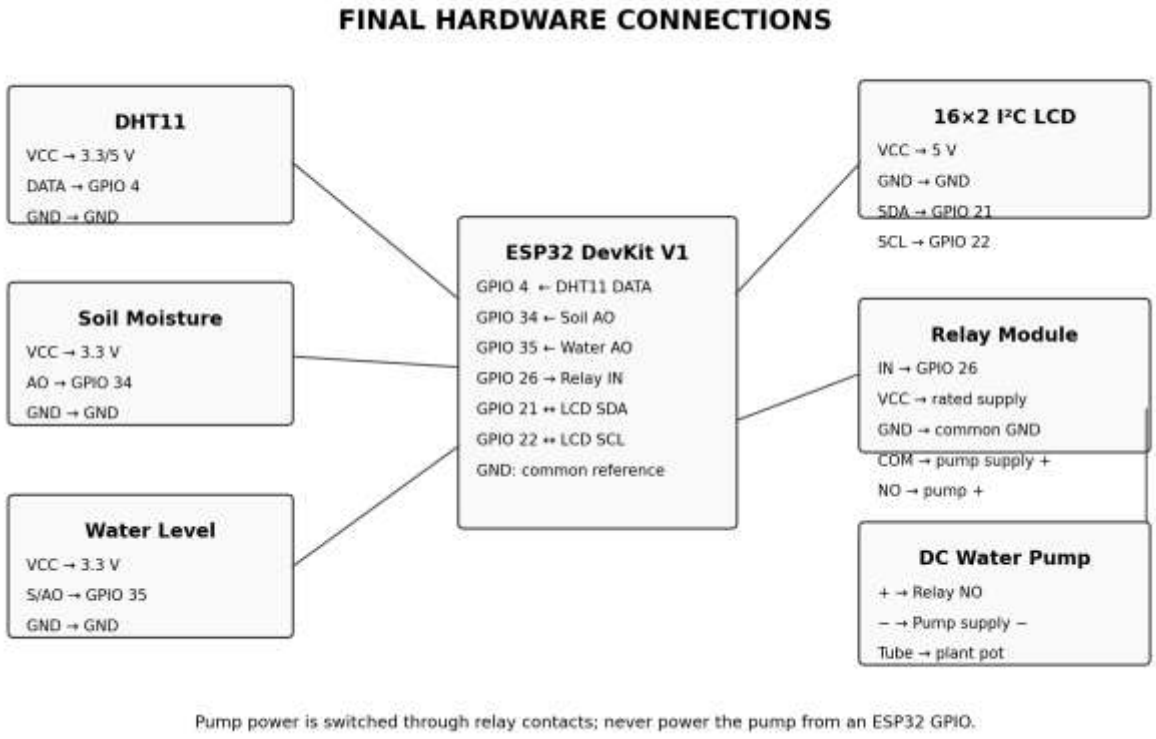


Figure 4.1 Wiring diagram and final pin connections

4.2 Pin Connection Table

Module	Module Pin	ESP32 / Connection
DHT11	DATA	GPIO 4
DHT11	VCC / GND	3.3 V or 5 V / GND
Soil moisture sensor	AO	GPIO 34
Soil moisture sensor	VCC / GND	3.3 V / GND
Water-level sensor	S / AO	GPIO 35
Water-level sensor	VCC / GND	3.3 V / GND
16×2 I²C LCD	SDA	GPIO 21
16×2 I²C LCD	SCL	GPIO 22
16×2 I²C LCD	VCC / GND	5 V / GND
Relay module	IN	GPIO 26
Relay contacts	COM	Pump supply +

Relay contacts	NO	Pump +
DC pump	Negative	Pump supply −

4.3 Hardware Integration

The sensors are connected first and verified individually before the pump circuit is integrated. This staged approach reduces troubleshooting complexity. The LCD and DHT11 are tested first, followed by soil-moisture calibration, water-level calibration, relay switching and finally pump operation. After all individual modules operate correctly, the final firmware combines them with the Wi-Fi dashboard.

The relay contacts are wired in normally-open configuration so that the pump remains OFF when the controller starts, resets, or loses power. The pump is powered from its own suitable supply path through the relay contacts. The electronics are mounted away from the water outlet and water tank. Cable ties and fixed routing are used to prevent the sensor wires from being pulled during demonstration.

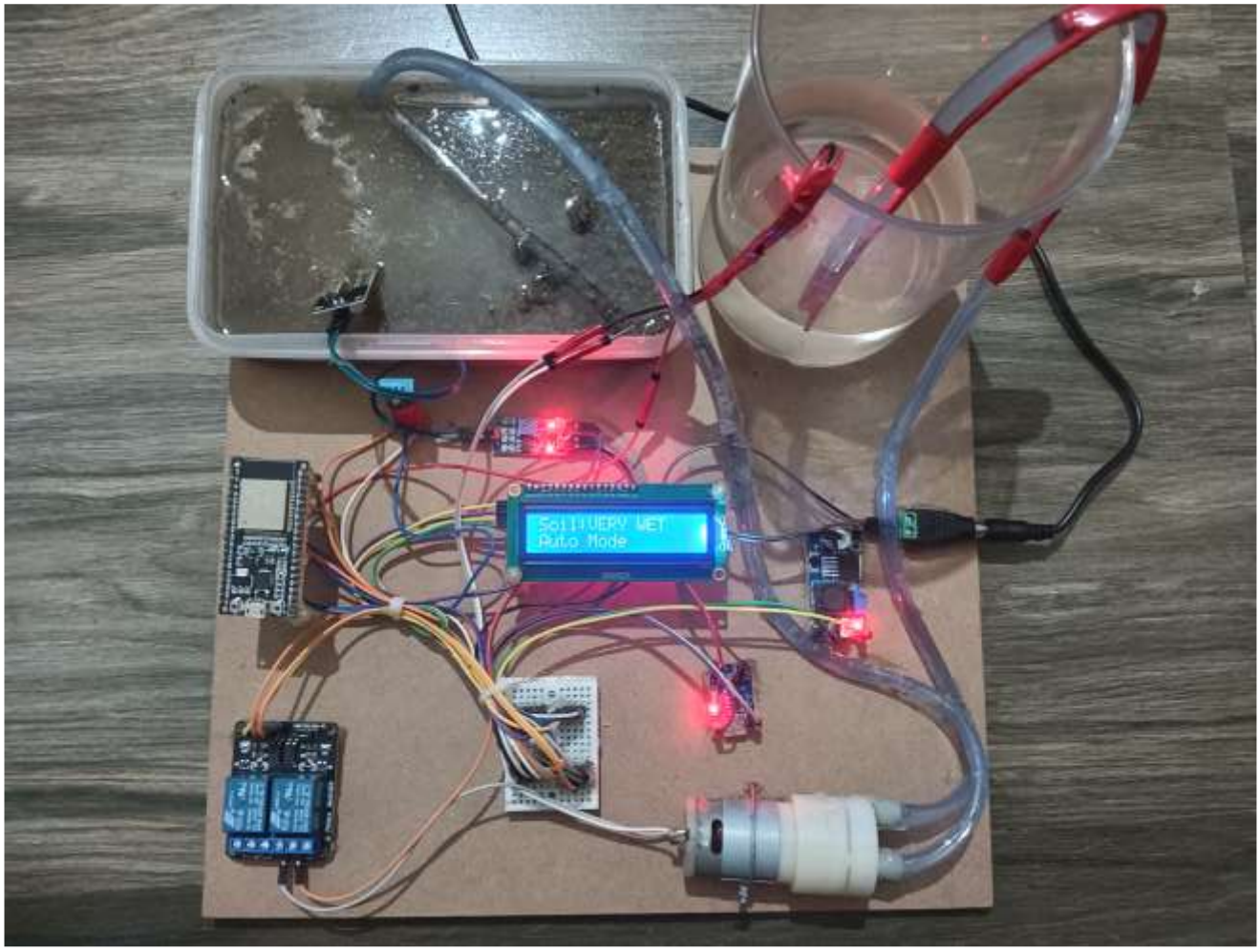


Figure 4.2 Representative final plywood-mounted prototype arrangement

4.4 Safety Considerations

- Do not connect the pump directly to an ESP32 GPIO pin.
- Use a pump supply that matches the pump rating.
- Keep the water tank and outlet tubing physically separated from the controller and exposed connections.
- Keep the pump OFF if the water-level sensor indicates an empty tank.
- Use the 30-second runtime limit as a backup against abnormal continuous pumping.
- Power OFF the circuit before changing high-current pump or relay wiring.

CHAPTER 5

SOFTWARE DESCRIPTION

The software integrates sensor acquisition, signal conversion, automatic decision-making, relay control, LCD presentation, Wi-Fi access-point operation and web-dashboard serving. All functionality is contained in a single ESP32 Arduino sketch so that the complete project can be compiled and uploaded from Arduino IDE.

5.1 Arduino IDE

Arduino IDE is used to write, compile, upload and debug the ESP32 firmware. The ESP32 board support package provides the required compiler, upload tools and Wi-Fi libraries. The Serial Monitor at 115200 baud is used during calibration and testing to observe raw soil values, moisture percentage, estimated pH, DHT11 values, raw tank values, tank state and pump state.

Typical development procedure: connect the ESP32 using a USB data cable, select the correct ESP32 board and serial port, install the required libraries, compile the sketch, upload it, open Serial Monitor at 115200 baud, and test all operating conditions.

5.2 Required Libraries

Wire.h

Provides I²C communication used by the LCD interface.

LiquidCrystal_I2C.h

Controls the 16×2 I²C LCD at address 0x27.

DHT.h

Reads temperature and humidity values from the DHT11 sensor.

WiFi.h

Configures the ESP32 as a local Wi-Fi access point.

WebServer.h

Implements the HTTP server used for the dashboard and JSON sensor endpoint.

5.3 Local Web Dashboard

The ESP32 runs in Wi-Fi Access Point mode and creates the network SmartWatering. This method is suitable for project demonstrations because it does not require a router, internet connection or cloud account. The dashboard is served directly from the ESP32. The root page contains HTML, CSS and JavaScript stored in program memory, while a /data endpoint returns current sensor and pump values in JSON format. JavaScript requests fresh data approximately every 1.5 seconds and updates the dashboard without reloading the entire page.

The dashboard contains status cards for soil moisture, estimated pH, temperature, humidity, soil condition, water tank and pump. It also shows automatic-control thresholds, network information, a live connection indicator and the time of the most recent update. Dashboard design changes do not alter the automatic watering logic.

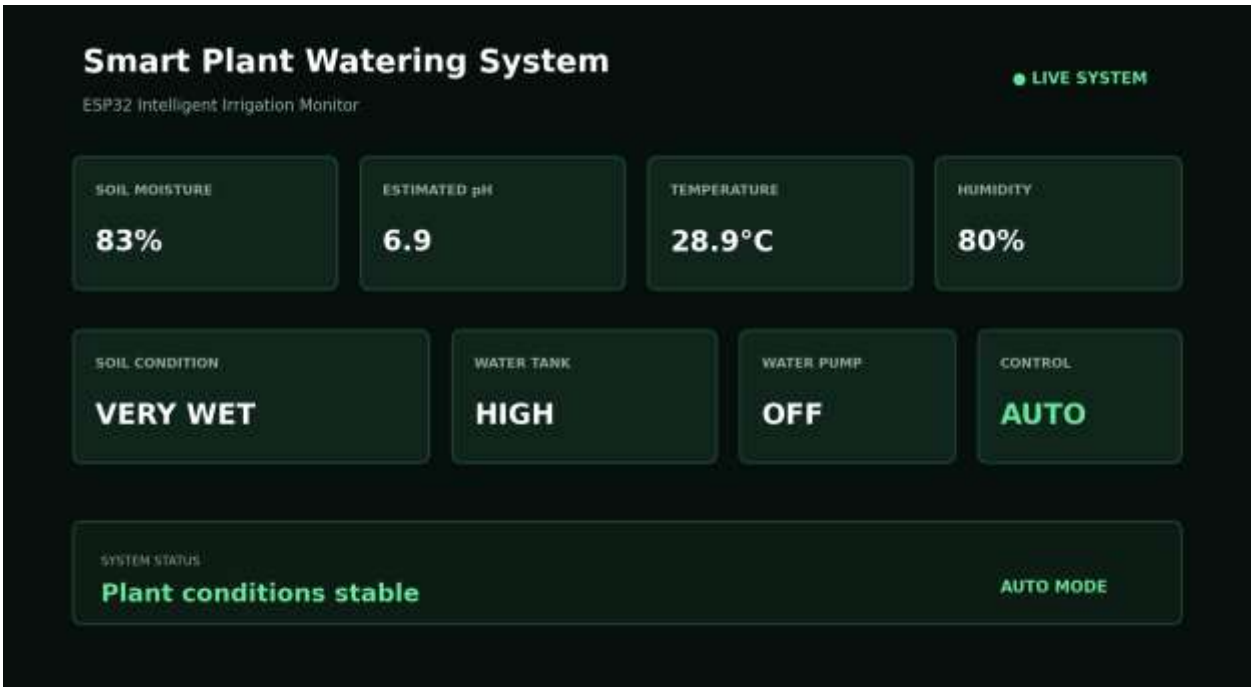


Figure 5.1 Representative ESP32 local web dashboard

5.4 LCD User Interface

During startup, the LCD first displays the project startup message, then the Wi-Fi name SmartWatering, and finally the dashboard IP. The IP remains visible until a browser opens the dashboard. After connection, the display rotates through four pages: moisture and estimated pH; temperature and humidity; tank and pump status; and soil condition with automatic-mode or no-water indication.

CHAPTER 6

SOFTWARE IMPLEMENTATION

The software implementation uses periodic sensor updates and a non-blocking web-server loop so that the dashboard remains responsive while automatic watering continues. Sensor values are averaged to reduce noise, mapped into useful engineering states, and evaluated by a simple state-based control algorithm.

6.1 Flowchart of Smart Plant Watering System

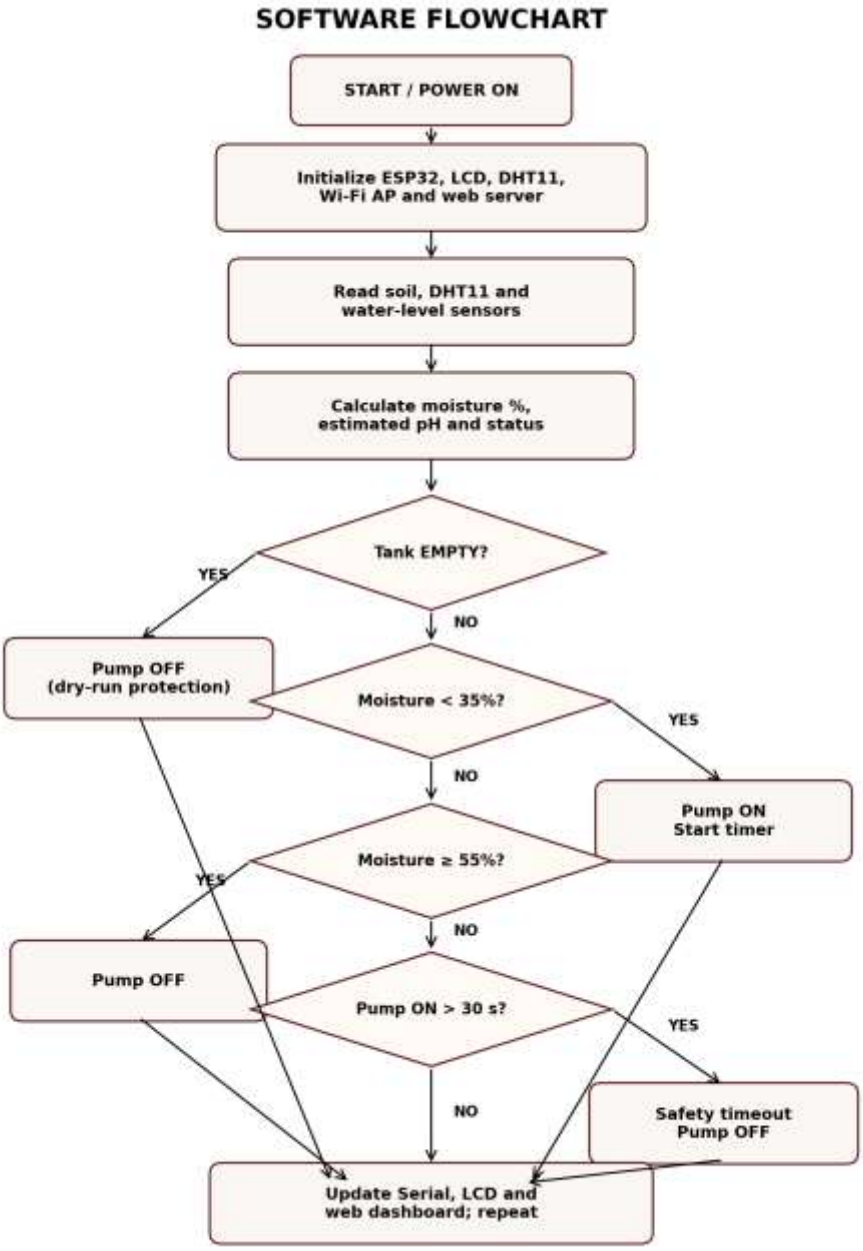


Figure 6.1 Flowchart of automatic watering and monitoring logic

6.2 Algorithm

1. Start the system and power ON the ESP32.
2. Initialize Serial communication, I²C LCD, DHT11 and relay output.
3. Keep the pump OFF during startup.
4. Create the local Wi-Fi access point named SmartWatering and start the web server.
5. Display the Wi-Fi name and dashboard IP address on the LCD.
6. Read ten soil-moisture samples and calculate the average raw value.
7. Map the averaged soil value from the calibrated 4095–1500 range to 0–100% moisture.
8. Assign an estimated demo pH from the moisture range: 6.4, 6.5, 6.7, 6.8 or 6.9.
9. Read temperature and humidity from the DHT11 and verify that the values are valid.
10. Read ten water-level samples and calculate the average raw value.
11. Classify tank state: EMPTY <200, LOW <1200, MEDIUM <2000, otherwise HIGH.
12. If the tank is EMPTY, force the pump OFF.
13. If the tank contains water and moisture is below 35%, switch the pump ON and start the runtime timer.
14. If moisture reaches 55% or above, switch the pump OFF.
15. If the pump remains continuously ON for 30 seconds, switch it OFF as a safety timeout.
16. Update Serial Monitor values, JSON dashboard data and the LCD pages.
17. Keep the web server active and repeat the control loop continuously.

6.3 Soil-Moisture Conversion

The project uses the experimentally determined constants SOIL_DRY = 4095 and SOIL_WET = 1500. The ESP32 map function converts the raw ADC value into a 0–100% scale, then constrains the result so that values outside the calibration endpoints do not produce percentages below 0 or above 100. The scale is used for control and user presentation and should not be interpreted as a laboratory-grade volumetric water-content measurement.

6.4 Estimated pH Demonstration Logic

Moisture Range	Displayed pH(est)	Interpretation
0–20%	6.4	Very dry
21–40%	6.5	Dry
41–60%	6.7	Normal
61–80%	6.8	Moist
81–100%	6.9	Very wet

The pH value above is not measured by the soil-moisture sensor. It is a demonstration parameter used because a dedicated pH probe was not included in the low-cost prototype. The LCD and web dashboard

therefore identify it as pH(est). A real deployment that requires soil chemistry information should use a dedicated pH electrode and suitable interface circuit.

6.5 Automatic Pump Control

The use of two moisture thresholds creates hysteresis. The pump starts only below 35% and does not stop until moisture reaches 55% or above. This prevents rapid ON/OFF switching when the reading fluctuates around a single threshold. Tank-empty protection has the highest priority and overrides the moisture condition. The maximum runtime protection provides another independent shutdown path.

CHAPTER 7

EXPERIMENTAL RESULTS

The complete prototype was tested in stages and then as an integrated system. Each sensor was first observed through Serial Monitor before being combined with the LCD, relay, pump and web dashboard. The following results are based on values measured during the actual prototype testing performed during development.

7.1 DHT11 Test Result

Parameter	Observed Value	Result
Temperature	28.9–29.3 °C	Stable reading
Humidity	78–80%	Stable reading
Interface	GPIO 4	Working

The DHT11 produced repeated temperature and humidity values without invalid readings during the reported test interval. These values were displayed on the LCD and dashboard and printed to the Serial Monitor.

7.2 Soil-Moisture Calibration Result

Condition	Raw ADC	Mapped Moisture	pH(est)
Sensor dry / out of soil	4095	0%	6.4
Dry/intermediate test	3253	32%	6.5
Wet soil	2128	75%	6.8
Wet soil	2087	77%	6.8
Wet soil	1920	83%	6.9
Wet soil	1782	89%	6.9

The soil sensor responded strongly to wet soil while producing the ESP32 ADC maximum of 4095 in the dry condition. The observed direction is therefore lower raw value for wetter soil and higher raw value for drier soil. Sudden changes during testing were associated with physically removing and reinserting the sensor, rather than with instability while the sensor remained in one position.

7.3 Water-Level Sensor Test Result

Test Condition	Observed Raw Range	Classified State
Completely dry	0	EMPTY
Residual droplets after removal	1041–1175	LOW
Deep in water	2219–2665	HIGH

The water-level sensor returned to zero after the residual water on its sensing tracks dried. The project therefore uses status bands rather than attempting to present the sensor as a precision percentage-volume measurement.

7.4 Relay and Pump Operation

Test	Expected Behaviour	Observed Result
Relay test	Click ON/OFF under GPIO 26 control	Working
Pump test	Pump follows relay state	Working
Dry soil + water available	Pump ON	Working
Moisture ≥55%	Pump OFF	Working
Tank EMPTY	Pump remains OFF	Working
30 s safety timeout	Pump forced OFF	Implemented

The complete automatic watering sequence operated correctly. When the soil reading was low enough to produce a moisture value below 35% and the tank contained water, the relay activated and the pump supplied water through the tube. When the sensor was placed in sufficiently wet soil and the mapped moisture rose above 55%, the relay was released and the pump stopped. When the tank sensor indicated EMPTY, the pump remained OFF even when the soil condition requested watering.

7.5 LCD and Dashboard Result

The LCD displayed the SmartWatering Wi-Fi name and dashboard IP during startup. After the browser connected to the local dashboard, the LCD switched to rotating sensor pages. The web dashboard opened through the ESP32 local IP and displayed live moisture, estimated pH, temperature, humidity, soil condition, tank status and pump status. Because the ESP32 itself hosts the page, the demonstration remained functional without an internet connection.

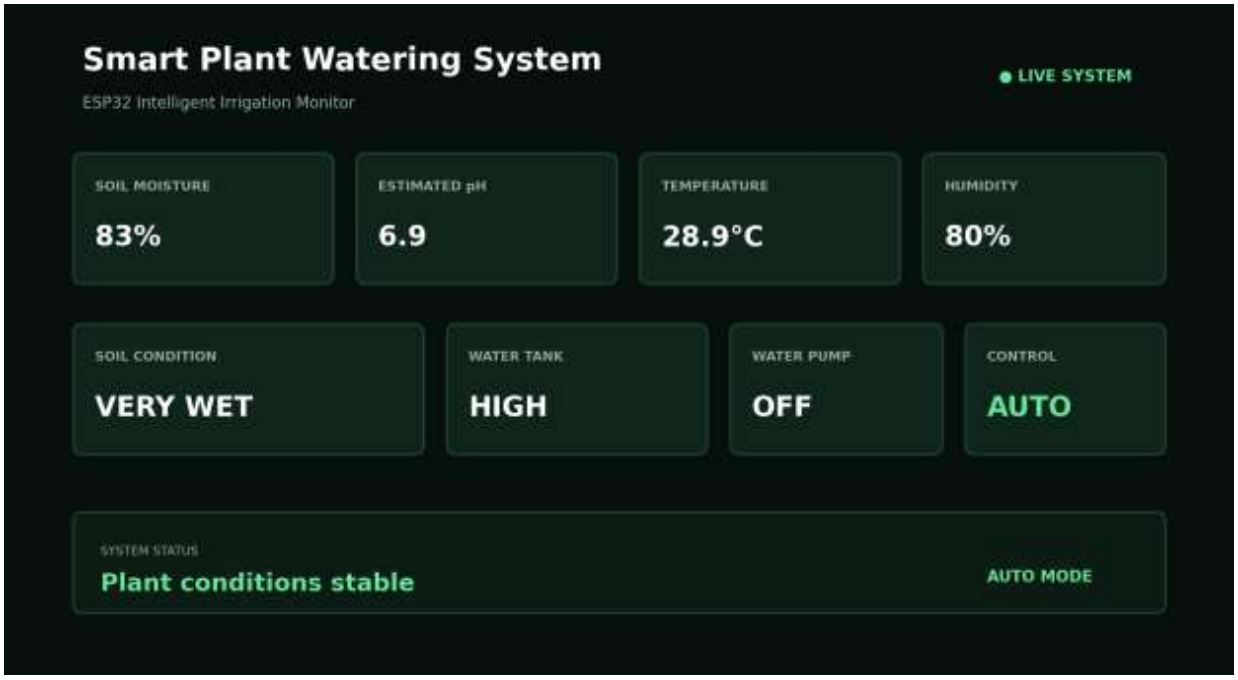


Figure 7.1 Dashboard monitoring result (representative view)

7.6 Overall Result

The final prototype successfully achieved the intended functions of automatic irrigation, sensor monitoring, tank protection, local LCD indication and local wireless monitoring. The use of calibrated thresholds and hysteresis produced predictable pump behaviour suitable for a college demonstration. The project also demonstrates the practical integration of analog sensing, digital sensing, I²C display communication, relay control, Wi-Fi networking and embedded web development in a single ESP32 system.

CHAPTER 8

APPLICATIONS, ADVANTAGES AND DISADVANTAGES

8.1 Applications

1. Home Gardens

- Automatic watering of indoor and outdoor potted plants when the user is busy or away.

2. Nurseries

- Low-cost monitoring and watering assistance for groups of small plants.

3. Balcony and Terrace Gardens

- Useful where water can be stored in a small tank and distributed to selected pots.

4. Educational Laboratories

- Demonstrates sensor interfacing, ADC conversion, I²C, Wi-Fi, embedded web servers, relays and automation.

5. Small Greenhouses

- Can be extended to multiple moisture zones, pumps or valves.

6. Assistive Plant Care

- Reduces the need for frequent manual checking and watering for users who have difficulty performing regular plant-care tasks.

8.2 Advantages

- Automatic watering based on measured soil condition rather than a fixed timer.
- Water conservation by stopping irrigation after adequate moisture is reached.
- Dry-run protection when the water tank is empty.
- Low-cost implementation using commonly available ESP32 modules.
- Local LCD monitoring without requiring a phone.
- Local Wi-Fi dashboard without internet or cloud dependency.
- Hysteresis control reduces frequent relay switching.
- Modular design allows sensors or actuators to be upgraded later.
- The dashboard makes the project easy to demonstrate and understand.
- The 30-second safety timeout reduces the risk of uncontrolled continuous pumping.

8.3 Disadvantages

- The soil-moisture percentage is based on prototype calibration and is not a laboratory measurement of volumetric water content.
- The water-level sensor is resistive and can show temporary readings when droplets remain on its sensing tracks.
- DHT11 has limited accuracy and response compared with higher-grade environmental sensors.

- The local web dashboard is accessible only while the user is connected to the ESP32 access point; it is not a cloud remote-monitoring system.
- A dedicated pH sensor is not used, so pH(est) must not be interpreted as real soil-pH measurement.
- A single soil sensor represents only the local area around the probe and may not represent moisture throughout a large pot or field.
- Long-term outdoor use would require waterproof enclosures, corrosion-resistant sensors and more robust power protection.

CHAPTER 9

CONCLUSION AND FUTURE SCOPE

9.1 Conclusion

The Smart Plant Watering System Using Sensor Technology was successfully designed, implemented and tested as a complete ESP32-based prototype. The system measures soil moisture, temperature, humidity and tank water level, then automatically controls irrigation using a relay-driven DC pump. The experimentally calibrated soil sensor, water-level thresholds and hysteresis-based pump control provide stable behaviour during demonstration. Pump activation below 35% moisture and deactivation at 55% or above prevent unnecessary relay switching, while tank-empty protection and a 30-second maximum runtime improve operational safety.

The LCD and local web dashboard make the system easy to monitor. The ESP32 creates its own SmartWatering Wi-Fi network, displays the dashboard IP on the LCD, and serves a responsive interface that can be opened from a phone or laptop without internet access. The completed prototype therefore demonstrates the integration of embedded sensing, automatic control, local user interfaces and IoT-style monitoring in a low-cost system. The project meets its main objective of reducing manual effort and unnecessary watering while maintaining a simple and understandable architecture suitable for educational and small-scale plant-care applications.

The prototype also clearly distinguishes between measured parameters and demonstration-only information. Soil moisture, temperature, humidity and tank status are sensor-derived, while the pH(est) value is only a mapped estimate used for presentation because a dedicated pH probe was not included. This distinction is important for technically correct interpretation of the system.

9.2 Future Scope

- Integrate a dedicated soil-pH electrode and signal-conditioning circuit for actual pH measurement.
- Replace the basic water-level strip with a more durable non-contact, ultrasonic or pressure-based tank-level sensor.
- Use a capacitive moisture sensor with improved waterproofing and long-term calibration.
- Add cloud connectivity or MQTT so the plant can be monitored remotely over the internet.
- Add a mobile application for historical graphs, notifications and manual override with safety interlocks.
- Store sensor values in a database for daily and weekly irrigation analysis.
- Add multiple soil sensors and multiple valves to support several pots or irrigation zones.
- Integrate weather information so outdoor irrigation can be reduced before expected rainfall.
- Add solar charging and battery monitoring for standalone outdoor operation.
- Use a higher-accuracy sensor such as DHT22, SHT31 or BME280 for environmental measurements.
- Add configurable thresholds through the dashboard while storing safe defaults in non-volatile memory.

- Improve the enclosure and PCB design for long-term field use.

REFERENCES

[1] J. Gutierrez, J. F. Villa-Medina, A. Nieto-Garibay and M. A. Porta-Gandara, “Automated Irrigation System Using a Wireless Sensor Network and GPRS Module,” IEEE Transactions on Instrumentation and Measurement, 2014.

[2] H. G. Jones, “Irrigation scheduling: advantages and pitfalls of plant-based methods,” Journal of Experimental Botany, 2004.

[3] Arduino Project Hub, examples and educational resources related to ESP32/Arduino smart irrigation and soil-moisture monitoring.

[4] A. G. Meti, K. S. Biradar, G. H. Manoj, P. N. Rayangoudra, V. Kumar and B. T. V. Murthy, “IoT and Solar Energy Based Multipurpose Agricultural Robot for Smart Farming,” IEEE ICDSIS, Hassan, India, 2022.

[5] Espressif Systems, ESP32 documentation for Wi-Fi, GPIO, ADC and embedded networking.

[6] Arduino documentation and library references for Wire, DHT sensor interfaces and I²C LCD operation.

APPENDIX A - FINAL DESIGN PARAMETERS

Parameter	Final Value
DHT11 pin	GPIO 4
Soil-moisture analog pin	GPIO 34
Water-level analog pin	GPIO 35
Relay input pin	GPIO 26
LCD SDA / SCL	GPIO 21 / GPIO 22
LCD I²C address	0x27
Soil dry calibration	4095
Soil wet calibration	1500
Pump ON threshold	< 35%
Pump OFF threshold	≥ 55%
Water EMPTY threshold	< 200
Water LOW threshold	< 1200
Water MEDIUM threshold	< 2000
Water HIGH threshold	≥ 2000
Maximum pump runtime	30 seconds
Wi-Fi SSID	SmartWatering
Typical AP dashboard IP	192.168.4.1